RAGCache++: Cache-Aware Document Ordering for Low-Latency RAG Serving

Kaizhen Tan [*1] **Rong Gu** [*1] **Mingyuan Li** [*1]

Abstract

Retrieval-Augmented Generation (RAG) improves factual grounding, but it also makes prompts much longer and increases prefill latency. Prefix caching in modern serving engines such as vLLM can reduce this cost, but it only works when different requests share the same token prefix. In practice, RAG queries often retrieve overlapping documents in different orders, so document overlap does not automatically become prefix overlap. We present RAGCache++, a lightweight prompt-layer method that reorders retrieved documents before inference. The method keeps a prefix tree over recently served document sequences and uses a greedy walk to place the most reusable prefix first, while leaving the serving engine unchanged. Across two vLLM configurations, our method reduces median TTFT by about 20%–33% compared with retrieval-order prefix caching, without hurting answer quality in our QA tests. We further show that the greedy policy reaches 97.5% of oracle ordering gain, suggesting that most reusable prefix locality can already be recovered with a simple scheduling layer between retrieval and serving.

1 Introduction and Motivation

Retrieval-Augmented Generation (RAG) improves factual grounding by adding external passages to the prompt before generation (Lewis et al., 2020; Gao et al., 2023). The price is higher serving latency. In many RAG systems, the retrieved context dominates the input length, so the prefill stage becomes the main bottleneck for interactive use.

Modern serving engines partly address this problem with prefix caching. Serving systems such as vLLM and SGLang can reuse previously computed KV states through Automatic Prefix Caching (APC) when a new request shares the same token prefix with an earlier one (vLLM Project, 2025; Zheng et al., 2024). This mechanism works well for highly repetitive prompts, but it is much less effective in RAG, where retrieved passages may overlap in content while appearing in different orders.

This mismatch is the central problem of this paper. Prefix caching works at the token level, while retrieval returns documents at the chunk level. Two consecutive queries may retrieve almost the same evidence set, but once the first document differs, the shared token prefix is broken and most reusable KV blocks are lost. In other words, document overlap is common in RAG, but prefix overlap is much rarer.

Our key idea is simple: instead of changing the serving engine, change the document order before the prompt is built. If retrieved documents are reordered to follow a recently cached sequence, the resulting prompt is more likely to match an existing prefix and benefit from APC. Based on this idea, we propose RAGCache++, a lightweight prompt-layer scheduler. It maintains a knowledge tree over recently served document sequences and uses a greedy walk to place the longest likely cached prefix first. The remaining documents are appended in retrieval order, so the system still falls back to the original ranking when reuse is unavailable.

Recent systems have shown that context order can affect cache reuse in long-context and RAG serving (Jin et al., 2026; Agarwal et al., 2025; Jiang et al., 2025). Our goal is therefore not to re-establish that ordering matters, but to ask how much of this benefit can already be recovered with a minimal and engine-agnostic design. In our setting, the core policy stays entirely at the prompt-construction layer, requires no modification to vLLM, and still performs close to exhaustive oracle ordering in our evaluation.

This paper studies three questions. First, can a pure prompt-layer policy provide meaningful TTFT gains in APC-based RAG serving? Second, is a greedy trie traversal sufficient, or does the problem require substantially more complex search? Third, do the gains remain visible beyond a single synthetic benchmark, under different overlap levels, workloads, and concurrent execution settings? Our results indicate that the answer is yes in all three cases.

[*]Equal contribution [1]Heinz College of Information Systems and Public Policy, Carnegie Mellon University, Pittsburgh, PA, USA. Correspondence to: Kaizhen Tan <kaizhent@cmu.edu>, Rong Gu <ronggu@andrew.cmu.edu>, Mingyuan Li <mingyua4@andrew.cmu.edu>.

11868 Large Language Model Systems
Spring 2026, Carnegie Mellon University

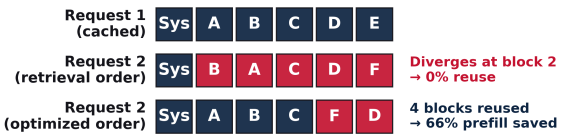

Figure 1. Prefix alignment in APC-based RAG serving. Request 1 is cached. Request 2 shares 60% of its documents but in retrieval order the prefix diverges at block 2, yielding 0% reuse. Reordering the shared documents to the front restores prefix alignment: 4 blocks reused, 66% prefill saved.

2 Related Work and Background

2.1 RAG Serving Pipeline

A standard RAG pipeline has four steps: query encoding, document retrieval, prompt construction, and LLM inference. In practice, prompt construction is cheap, while inference dominates latency. For this reason, the main serving problem is not how to concatenate the retrieved passages, but how to reduce repeated prefill work when many requests contain overlapping context.

2.2 vLLM Automatic Prefix Caching

vLLM manages KV cache memory in fixed-size blocks through PagedAttention (Kwon et al., 2023) and provides Automatic Prefix Caching (APC) on top of this design (vLLM Project, 2025). APC reuses cached KV blocks when a new request shares the same token prefix as an earlier one. The design documentation further explains that vLLM hashes each block together with its prefix history, so reuse is inherently prefix-sensitive.

This detail matters for RAG. Once an early block differs, the later blocks no longer line up with the cached chain, even if the two prompts still contain many of the same documents. Therefore, a small difference near the beginning of the prompt can remove most of the possible reuse.

Implication for RAG. In document retrieval, overlap is usually set-based, while APC requires ordered prefix agreement. If two requests retrieve the same documents in different positions, their token prefixes diverge immediately and reuse drops sharply. Figure 1 illustrates this gap and shows why reordering is a natural intervention point.

2.3 Related Work

Several recent papers study KV reuse in long-context or RAG inference, but they do so at different layers of the system.

RAGCache (Jin et al., 2026) is the closest work to ours. It also uses a tree structure over document sequences, but its design is tied to a richer cache-management stack inside the serving system. Our work borrows the basic intuition of sequence-aware reuse, but focuses on a smaller target: document ordering at the prompt layer, with no engine modification.

ContextPilot (Jiang et al., 2025) studies context reuse for long-context inference and proposes a context index, context ordering, and de-duplication. Its results support the same broader message as ours: prefill can often be reduced by reorganizing context before inference, rather than by redesigning the model itself.

Cache-Craft (Agarwal et al., 2025) shows that plain prefix caching can be weak in production RAG workloads because retrieved chunks and their relative order vary across requests. This directly motivates our setting. Instead of replacing prefix caching, we try to make more requests compatible with it.

TurboRAG (Lu et al., 2025) and **CacheBlend** (Yao et al., 2025) address a harder problem: KV reuse even when reusable chunks are not part of one contiguous prefix. These approaches are more general than ours, but also more involved. Our method is narrower, but it is easy to deploy because it works with existing APC mechanisms.

CacheGen (Liu et al., 2024) focuses on compressing and streaming KV caches efficiently. This is orthogonal to our setting. In principle, cache compression and cache-aware ordering could be combined in one serving stack.

3 Methodology

3.1 Overview

RAGCache++ sits between retrieval and prompt construction. Given a retrieved document set, it reorders the documents so that the final prompt is more likely to share a long prefix with recently served prompts. The serving engine still performs all KV-cache allocation and reuse; our layer only decides the document order.

3.2 Knowledge Tree

The knowledge tree is a trie whose root-to-leaf paths record document orders that have recently appeared in served requests. Each node stores a document ID together with lightweight metadata about recent use.

The tree supports two basic operations. First, after a request finishes, its ordered document sequence is inserted into the trie. Second, when a new request arrives, the trie is consulted to estimate which document order is most likely to align with a cached prefix. The tree therefore acts as a compact memory of recent prompt structure, rather than as a replacement for vLLM's own KV cache.

Algorithm 1 Cache-aware document ordering

Require: Retrieved doc IDs $D = \{d_1, \ldots, d_k\}$ (in retrieval rank), knowledge tree T
Ensure: Ordered sequence O

```
O ← [], R ← D, node ← T.root
while R ≠ ∅ do
    found ← false
    for each d ∈ R do
        c ← node.getChild(d)
        if c ≠ null and c.isCached() then
            Append d to O; remove d from R
            node ← c; found ← true; break
        end if
    end for
    if found = false then
        Append remaining R in retrieval-rank order to O
        break
    end if
end while
return O
```

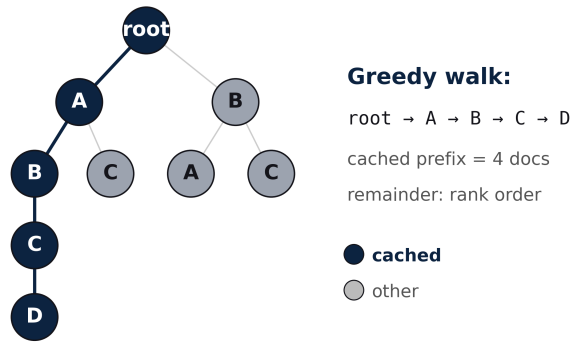

Figure 2. Knowledge tree example. Dark nodes are cached; gray nodes are evicted or from other branches. Given candidate set $\{A, B, C, F, D\}$, the greedy walk follows root→A→B→C→D (the longest cached path), yielding a 4-document prefix hit. The remaining document (F) is appended in retrieval-rank order.

3.3 Greedy Ordering Algorithm

Our ordering policy is shown in Algorithm 1. Starting from the trie root, the algorithm repeatedly asks whether one of the remaining retrieved documents extends a cached path. If yes, that document is placed next in the prompt. If no cached continuation exists, the remaining documents are appended in their original retrieval order.

This policy is intentionally simple. It does not try to solve a full combinatorial search problem at runtime. Instead, it greedily extends the longest usable prefix it can find from the current trie state. Figure 2 shows the intuition.

The objective behind this policy is straightforward. APC benefits grow with the length of the shared prefix, so we want the reordered prompt to match as many leading documents of a cached sequence as possible. The greedy walk is inexpensive for typical top-k values, and our later experiments show that it is already very close to oracle ordering.

3.4 Knowledge Tree as an Approximate Cache Index

The knowledge tree is only an approximation of the actual APC state. It records what has been served recently, not what remains resident in GPU memory. This distinction matters because vLLM controls block eviction internally. As a result, the trie can sometimes predict reuse for a path whose KV blocks have already been evicted.

We treat this mismatch as a systems issue rather than a modeling failure. In stable sequential runs, the approximation is often good enough by itself. Under tighter memory or more concurrent execution, we add a lightweight feedback mechanism to detect stale paths and prune them.

3.5 Integration with vLLM

The integration with vLLM has three pieces: a prompt builder that accepts retrieved documents and applies the ordering policy, a knowledge tree maintained at the application layer, and ordinary `LLM.generate()` calls with APC enabled. No internal vLLM symbols are required. This is important for portability, because it keeps the method close to a drop-in middleware component rather than a custom fork of the serving engine.

3.6 Cache-State Feedback Loop

To reduce trie staleness, we use a TTFT-based feedback loop. The intuition is that if the trie predicts a long reusable prefix but the observed TTFT is still close to a cold request, then the supposed cached path is probably no longer resident. In that case, we mark the path as stale and stop relying on it.

This feedback loop closes the gap between logical reuse and physical cache residency. The trie proposes a likely reusable ordering, vLLM executes the request, and the observed TTFT is used to decide whether the trie should trust that path in the future. The mechanism is simple, but it lets the prompt-layer policy adapt online without direct instrumentation of vLLM internals.

3.7 Retrieval-Inference Pipelining

Besides document ordering, we also implement a secondary optimization that overlaps retrieval with a small warmup request. The system prompt is identical across requests, so its KV states can be prefetched while the retriever is still working. When retrieval finishes, the full request can reuse

this shared prefix immediately.

This pipelining idea is orthogonal to our main contribution. It does not change the ordering policy, and it mainly helps cold starts or recovery after eviction. We include it because it fits the same design philosophy: reduce prefill waste without changing the serving engine itself.

4 Implementation Details and Code Snippets

RAGCache++ is implemented as a Python package on top of vLLM and FastAPI. The implementation has two layers. The first is the minimal ordering path, which contains the trie walk and prompt construction logic used in the main experiments. The second is a fuller proxy system that adds cache-state bookkeeping, feedback, and service integration. This split is useful in practice, because it separates the core idea from the engineering needed for a more complete serving prototype.

4.1 Module Organization

Table 1 summarizes the main modules. The cache-related components and the prompt builder are decoupled, so the ordering logic can be tested without running the full GPU serving stack.

Table 1. Core-library modules (excludes benchmark scripts, workload generators, and evaluation harnesses). Cache subsystem and integration layer are decoupled: the trie + prompt builder can run without GPU for unit tests.

Module	LOC	Role
`cache/knowledge_tree.py`	232	Trie + KV metadata
`cache/cache_manager.py`	615	Multi-tier accounting
`cache/pgdsf_policy.py`	153	PGDSF eviction
`cache/spatial_index.py`	317	Geohash spatial filter
`vllm_integration/prompt_builder.py`	107	Greedy ordering + prompt
`vllm_integration/serving.py`	296	Proxy + feedback + API
`serving/rag_controller.py`	361	Pipelining controller
`config.py`	102	Config dataclasses

4.2 Key Code Snippets

Greedy trie-based ordering. The policy itself is small. It walks the knowledge tree, extends a reusable prefix when possible, and otherwise falls back to retrieval order.

Listing 1. Greedy ordering (`prompt_builder.py`). `remaining` is a `set`; when multiple cached children exist at one trie level the iteration order is Python-hash-dependent (ties are rare in practice—the trie typically has one matching child per level). The cold-path loop appends every remaining document in retrieval-rank order (matching Algorithm 1).

```
def optimize_doc_order(doc_ids, kt):
    ordered, remaining = [], set(doc_ids)
    node = kt.root
    while remaining:
        found = False
        # Hot path: extend the cached prefix by one
    cached child
        for d in list(remaining):  # set order; tie-
    break is hash-dependent
            child = node.get_child(d)
            if (child is not None
                and child.kv_metadata is not None
                and child.kv_metadata.tier != "none"):
                ordered.append(d); remaining.discard(d
    )
                node = child; found = True
                break   # only break the inner for;
    outer while continues
        if not found:
            # Cold path: append ALL remaining docs in
    retrieval-rank
            # order, then exit the outer while loop.
            for d in doc_ids:
                if d in remaining:
                    ordered.append(d)
            break
    return ordered
```

TTFT-based mismatch detection. Because the trie only approximates the actual cache state, the system uses TTFT observations to detect stale paths and prune them when needed.

Listing 2. Cache-state feedback (`serving.py`).

```
def check_mismatch(self, predicted_prefix_len, k,
     ttft_ms):
    self.total_observations += 1
    pred_reuse   = predicted_prefix_len / max(k, 1)
    actual_reuse = max(0.0, 1.0 - ttft_ms / max(self.
     cold_ttft, 1.0))
    self.history.append((pred_reuse, actual_reuse))
    if pred_reuse > 0.3 and actual_reuse < 0.15:
        self.mismatches += 1
        return True              # trie thinks hit,
     vLLM evicted
    return False
```

Integrated serve loop. In the full proxy, ordering, prompt build, inference, feedback, and admission are connected in a single request path.

Listing 3. `VLLMCacheProxy.serve_request` (abridged).

```
def serve_request(self, query, doc_ids, doc_contents,
     max_tokens=1):
    # 1. Greedy trie walk -> ordered ids + predicted
     prefix length
    ordered, predicted = self._greedy_walk(doc_ids)
    # 2. Build prompt
    prompt, _ = build_rag_prompt(query, ordered,
     doc_contents)
    # 3. vLLM inference (APC enabled at LLM
     construction)
    t0 = time.perf_counter()
    out = self.llm.generate([prompt], sp)
    ttft_ms = (time.perf_counter() - t0) * 1000
    # 4. Feedback loop
    self.feedback.update_cold_estimate(ttft_ms,
     predicted, len(doc_ids))
    miss = self.feedback.check_mismatch(predicted, len
     (doc_ids), ttft_ms)
    # 5. Prune stale paths if vLLM evicted them
    if miss:
        self._prune_path(ordered[:predicted])
    # 6. Admit into multi-tier cache state
    metas = [self.cache_manager.admit(d, 200,
     retrieval_rank=i+1)
```

```
            for i, d in enumerate(ordered)]
    self.cache_manager.insert_sequence(ordered, metas)
    self.cache_manager.step()
    return {"ttft_ms": ttft_ms, "predicted_prefix_len"
     : predicted, ...}
```

The practical point is that the optimization does not require deep changes to the serving engine. Existing RAG applications can adopt the method by adding a thin middleware layer around the standard inference call.

4.3 Engineering Details and Trade-offs

Several engineering choices are worth noting. First, we use the public vLLM API rather than private engine internals, which makes the integration easier to maintain across versions. Second, the fallback behavior is simple and safe: if the trie is empty or no cached continuation is found, the system returns to retrieval order. Third, the host-side data structure is lightweight, so the method does not trade lower TTFT for a large memory footprint outside the GPU.

Overall, the implementation reflects the same design goal as the method itself: keep the core mechanism small, keep the deployment path simple, and let the serving engine continue to handle actual KV management.

5 Experiments

5.1 Experimental Setup

Hardware. We evaluate on two vLLM configurations: a smaller setup on RTX 4060 Ti with Qwen2.5-1.5B-Instruct, and a larger setup on RTX 4090 with Qwen2.5-7B-Instruct (Yang et al., 2024). Full software and hardware details are listed in Appendix A.

Workload. Our main benchmark uses a controlled synthetic corpus where consecutive queries within a burst retrieve overlapping documents. This setup is intentionally simple. It isolates the systems question of interest: when document overlap exists, can reordering turn that overlap into APC-friendly prefix reuse?

Strategies. We compare four settings: no cache, APC with retrieval order, APC with lexicographic sorting, and APC with our optimized ordering. Each setting is run with a fresh model instance to avoid contamination from previous cache state.

Metrics. We focus on TTFT, since document ordering mainly affects prefill rather than decode. We also report a cache-hit proxy, but we treat TTFT as the primary metric because it reflects both how often reuse happens and how deep the reusable prefix is.

Table 2. TTFT comparison across strategies and hardware. Best APC result in **bold**.

GPU	Strategy	p50	p95	Mean	Fast%†
4060 Ti	No Cache	158.2	161.4	158.3	0.0
	APC+Retrieval	72.5	159.9	69.9	17.9
	APC+Sorted	108.0	164.9	101.7	25.3
	APC+Optimized	**57.9**	**161.3**	**68.1**	18.9
4090	No Cache	121.3	123.7	121.4	0.0
	APC+Retrieval	59.6	117.4	55.9	12.3
	APC+Sorted	81.2	118.7	76.3	25.6
	APC+Optimized	**40.1**	**117.4**	**51.4**	19.0

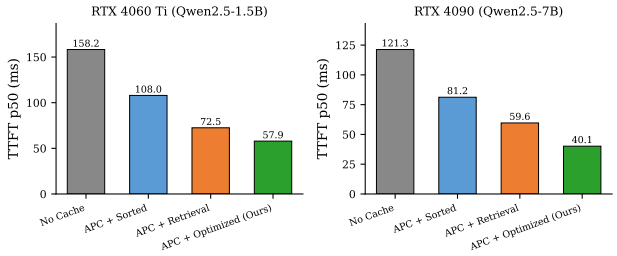

Figure 3. TTFT p50 comparison across strategies. Optimized ordering achieves the lowest latency on both GPUs despite not having the highest Fast%† proxy.

5.2 Main Results

Table 2 and Figure 3 show the main result: optimized ordering consistently lowers median TTFT on both hardware configurations.

Two observations are more important than the raw numbers. First, the gain is concentrated at p50 rather than p95. This is expected, because tail cases are often cold requests where no ordering policy can create reuse. Second, our method beats simple sorting even though sorting can show a higher hit-rate proxy. This tells us that not all hits are equally useful. A few long prefix matches matter more than many shallow ones.

Hit rate vs. latency. Figure 4 makes this point more clearly. The sorted baseline creates more fragmented matches, while our policy tries to preserve deeper contiguous prefixes. For APC-based serving, deeper reuse is the more valuable resource.

6 Analysis, Ablations, and Findings

6.1 Overlap Sensitivity

We next vary the amount of document overlap in the trace. The main pattern is intuitive: when overlap is moderate, ordering helps the most; when overlap is extremely high, retrieval order is already close to good enough; when over-

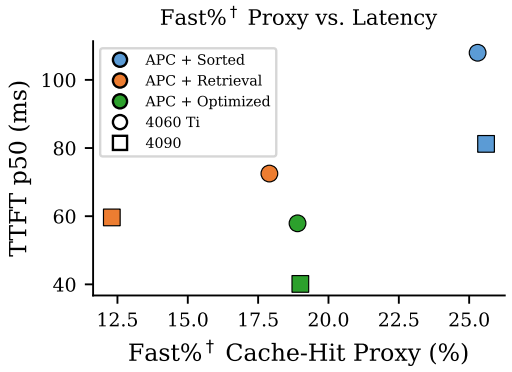

Figure 4. Fast%† cache-hit proxy vs. TTFT across strategies and GPUs. A higher proxy value does not guarantee lower latency—prefix *length* matters more than hit *count*.

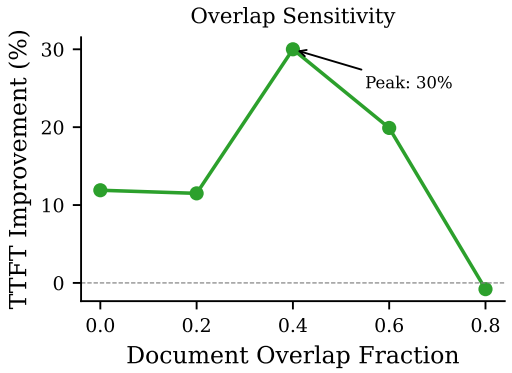

Figure 5. TTFT improvement vs. *nominal* overlap fraction (the parameter fed to the trace generator). Note: the x-axis is the generator parameter, not the actual Jaccard; the 0.0 and 0.2 points both produce an actual Jaccard of ≈0.39 due to the generator's minimum-sharing floor (see Section 5 and Table 14).

lap is truly absent, there is very little reusable structure to exploit.

This result is useful because it clarifies the operating regime of the method. RAGCache++ is not a universal speedup for all workloads. It is most effective when requests are related enough to share evidence, but not so similar that the retriever already returns the same order.

6.2 Answer Quality Preservation

A natural concern is whether document reordering changes what the model answers. On our extractive QA setting, the answer does not change. This is consistent with the design of the method: we reorder evidence, but we do not remove any retrieved content.

Interpretation. For tasks where the answer is explicitly present in the retrieved passages, evidence order is usually less important than evidence availability. In this regime, reordering mainly changes prefill cost, not task information.

Table 3. Multi-hop reasoning: does ordering disrupt evidence chains? (Qwen2.5-7B, 40 two-hop QA pairs, 3 distractors each.)

Evidence	Doc Order	EM	Contains EM	F1
Natural	Retrieval	0.675	0.775	0.862
Natural	Optimized	0.675	0.775	0.862
Reversed	Retrieval	0.625	0.750	0.826
Reversed	Optimized	0.625	0.750	0.826

Table 4. Per-request latency breakdown (RTX 4090, 195 post-warmup queries). APC+Opt. Order+Build totals 66 μs (incremental ∼26 μs over APC+Retr.) while reducing inference p50 by 29%.

Strategy	Order	Build	Gen. (mean)	Gen. (p50)	OH
APC+Retr.	4.0 μs	36.3 μs	55.5 ms	58.6 ms	0.07%
APC+Opt.	22.2 μs	44.4 μs	52.5 ms	41.4 ms	0.13%

Multi-hop reasoning. We also test a small multi-hop setting, where the answer requires combining two pieces of evidence. Here the absolute difficulty depends on the evidence arrangement itself, but within each evidence arrangement, optimized ordering and retrieval ordering give the same quality.

This result is reassuring but should be interpreted carefully. It does not prove that order never matters for reasoning. It shows a narrower claim: in our test suite, the ordering policy did not introduce an observable quality drop.

6.3 Systems Analysis

The rest of this section asks whether the TTFT gains come with hidden systems costs. Our answer is largely no.

Latency profiling. Table 4 separates ordering, prompt construction, and model execution. The main message is simple: the added CPU-side work is tiny, while the inference stage becomes noticeably faster because more prefix blocks can be reused.

Throughput. For long-output workloads, decode dominates total time, so TTFT savings do not automatically translate into large throughput changes. Even so, optimized ordering matches baseline APC throughput closely, which means the method does not buy latency by quietly hurting overall serving capacity.

GPU memory. We do not observe a meaningful GPU-memory penalty from ordering itself. This is expected, because the method changes reuse patterns inside the existing APC pool rather than increasing the pool size. The knowledge tree lives in host memory and is small in our experiments.

Table 5. Throughput on RTX 4090 (Qwen2.5-7B, 200 queries, 50 output tokens each). tok/s counts total tokens processed (input prompt + output); e.g., sequential 1.19 req/s × ∼1,150 tokens/req ≈ 1,370 tok/s. Ordering matches baseline APC throughput (<0.4% difference).

	Sequential		Batched	
Strategy	**req/s**	**tok/s**	**req/s**	**tok/s**
No Cache	1.10	1,270	7.88	9,064
APC+Retrieval	1.19	1,370	82.05	94,382
APC+Optimized	1.19	1,371	81.75	94,029

Table 6. End-to-end pipelining: measured wall-clock TTFT (RTX 4090). Serial $= T_{\text{ret}} + T_{\text{cold}}$ with $T_{\text{cold}} = 56.8$ ms; Pipelined $= \max(T_{\text{ret}}, T_{\text{sys}}) + T_{\text{warm}}$ with $T_{\text{sys}} = 21.6$ ms and $T_{\text{warm}} = 28.2$ ms.

T_{ret} **(ms)**	**Serial (ms)**	**Pipelined (ms)**	**Saving**
20	76.8	50.4	34.4%
50	106.8	78.8	26.2%
100	156.8	128.8	17.9%

Pipelining analysis. The warmup-based pipelining design from Section 3 behaves as expected in end-to-end measurements. When retrieval is not negligible, overlapping retrieval with reusable system-prompt prefill produces additional savings.

Cache efficiency. The cache-efficiency analysis is consistent with the TTFT results. Optimized ordering not only creates reuse more often, but more importantly, creates reuse of greater depth.

Concurrent load. The gains also survive under concurrent execution. This matters because sequential traces can sometimes overstate locality. In our batched experiments, the improvement becomes smaller than in the single-request case, but it remains visible across all tested batch sizes.

Baseline comparison. We compare the greedy walk with several alternative orderings, including a recency heuristic and an exhaustive oracle. The greedy method is already very close to oracle, which suggests that expensive search is unnecessary for this problem size.

Robustness. Additional sweeps over cache pressure and context length show that the improvement is not tied to one narrow memory setting. The exact percentage changes from run to run, but the qualitative result remains stable.

Generality. We also test models outside one scale point and find that the benefit is not unique to a single model size. The magnitude changes, but the same ordering effect appears across the tested configurations.

Table 7. Cache efficiency: prefill saved per request (RTX 4090, 195 queries, No-Cache p50 = 116 ms in this run; differs from Table 2/9 due to independent benchmark runs per Section 5).

Strategy	**Saved (p50)**	**Saved (mean)**	**Saved (p95)**	**Benefit**
APC+Retr.	50.1%	52.8%	75.6%	176/195
APC+Opt.	66.3%	57.6%	77.7%	185/195

Table 8. Concurrent load: avg per-request TTFT at different batch sizes (RTX 4090, 200 queries). Ordering benefit persists across all concurrency levels.

Batch	**No Cache**	**APC+Retr.**	**APC+Opt.**	**Improv.**
1	114.7	54.7	50.7	7.3%
2	110.5	48.0	44.3	7.7%
4	103.4	41.6	37.5	9.9%
8	100.9	38.3	35.7	6.8%

Geospatial workload. To move beyond the main synthetic benchmark, we construct a geospatial workload inspired by Spatial-RAG and ItiNera (Yu et al., 2025; Tang et al., 2024). The overlap in this workload is lower and less controlled, so it is a useful stress test for whether the method still helps when reuse opportunities are weaker and more irregular.

The gain is smaller here than in the bursty synthetic benchmark, which is exactly what we would expect from the lower overlap level. Even so, the direction is still positive, showing that the method does not rely on one artificial trace shape.

6.4 Workload Diversity

We further evaluate three workloads with different overlap regimes. The goal is not to claim one universal number, but to show how the benefit changes with workload structure.

NQ-Open (high overlap, Jaccard = 0.88). Using a corpus derived from Natural Questions (Kwiatkowski et al., 2019), we obtain a very high-overlap trace. In this setting, APC in retrieval order is already strong, so our extra gain is small. This is consistent with the overlap-sensitivity result above.

Mixed-topic (moderate overlap, Jaccard = 0.31). With a trace that alternates between related and unrelated topics, the benefit becomes more visible again. This is the regime where ordering feels most natural: there is enough repeated evidence to exploit, but enough variation that retrieval order is no longer cache-friendly.

Wikipedia-style TF-IDF (low natural overlap, Jaccard = 0.06). Finally, we build a larger, more diverse corpus and use TF-IDF retrieval without manually controlling overlap.

Table 9. Ordering baselines (RTX 4090, 195 queries). All six strategies measured in a single consistent run with one shared trace. Greedy trie achieves 97.5% of oracle's TTFT reduction (+29.6% vs. oracle's +30.3%).

Strategy	p50 (ms)	Mean (ms)	vs. Retr.
No Cache	121.55	121.73	–
APC+Sorted	81.75	77.42	−35.5%
APC+Retrieval	60.34	57.63	baseline
APC+Recency	59.52	56.62	+1.4%
APC+Optimized (Ours)	**42.50**	**53.77**	+29.6%
APC+Oracle	42.04	53.08	+30.3%

Table 10. Geospatial workload results (RTX 4090, 200 queries, avg Jaccard = 0.17).

Strategy	p50 (ms)	p95 (ms)	Mean (ms)
No Cache	110.3	113.1	109.5
APC+Retrieval	96.5	103.1	96.1
APC+Optimized	**92.2**	**103.0**	**85.7**

Even though the mean overlap is low, optimized ordering still helps, suggesting that temporal locality inside the trace matters more than a single global Jaccard number.

6.5 End-to-End Pipeline Latency

In deployment, retrieval latency comes before inference. Table 12 therefore reports end-to-end TTFT with simulated retrieval delay. The improvement becomes smaller once retrieval takes a larger share of total time, but it remains visible, especially at lower batch sizes where prefill cost is less amortized.

6.6 System Validation

Our method relies on the trie as an approximation of real APC state, so we also check whether this approximation is reasonable. The observed correlation between predicted reuse and TTFT-implied reuse is high, which supports the basic design choice.

Integrated pipeline. When we evaluate the full proxy with feedback and pruning, the overall behavior remains close to the simpler ordering wrapper. In other words, the extra bookkeeping does not remove the benefit that motivates the method in the first place.

7 Conclusion and Discussion

We presented RAGCache++, a prompt-layer method for improving KV reuse in APC-based RAG serving. The main idea is to reorder retrieved documents so that the final prompt is more likely to match a recently cached prefix. This design is intentionally small. It does not require changes to the serving engine, and it still delivers consistent TTFT gains across our main benchmarks.

Table 11. Wikipedia-style corpus evaluation with TF-IDF retrieval (RTX 4090, 200 queries, 1,000 constructed passages / 20 topics; mean Jaccard = 0.06, no controlled overlap).

Strategy	p50 (ms)	p95 (ms)	Mean (ms)	vs. Retr.
No Cache	45.8	54.1	47.2	–
APC+Retrieval	33.9	46.8	34.2	baseline
APC+Optimized	**24.8**	**46.5**	**27.2**	+26.8%

Table 12. End-to-end TTFT (ms) including retrieval latency (RTX 4090, 200 queries; workload generates 50 output tokens per request but TTFT excludes the 50-token decode).

T_{ret}	Batch	APC+Retr.	APC+Opt.	Improv.
10 ms	1	69.6	51.8	25.6%
	4	51.8	46.9	9.5%
	8	48.9	45.9	6.1%
30 ms	1	89.7	72.4	19.3%
	4	71.8	67.8	5.6%
	8	68.8	66.8	2.9%
50 ms	1	109.3	92.2	15.6%
	4	91.8	87.4	4.8%
	8	88.7	86.3	2.7%

The main takeaway is straightforward. In APC-based RAG, cache efficiency is not decided only by the model or the serving backend. It is also shaped by how retrieved evidence is arranged before inference. From this perspective, document ordering is a lightweight scheduling problem between retrieval and serving. Our experiments show that even a simple greedy trie walk is already enough to recover much of the lost locality. The method stays close to oracle ordering, adds very little CPU overhead, and does not hurt answer quality in our QA tests.

At the same time, the method has clear limits. It works best when the workload has some temporal locality and moderate document overlap. If overlap is already extremely high, retrieval order is often good enough. If overlap is truly low, there is little reusable structure to exploit. Our evaluation is also still limited in scale. We do not test large multi-tenant traces or production traffic, and we only study a small set of models and retrieval settings.

There are several natural directions for future work. One is to combine prompt-layer ordering with position-insensitive reuse methods, so that both prefix-compatible and non-prefix-compatible sharing can be exploited. Another is to use more direct cache-state signals from the engine when they are available, instead of relying mainly on the trie as an approximation. A third is to evaluate the method on more realistic traces where locality comes from user behavior rather than from controlled synthetic bursts. Overall, we view this

work as a practical systems result: before redesigning the cache itself, it is already worth asking whether the prompt can be arranged in a more cache-friendly way.

8 Team Member Contributions

Kaizhen Tan. Led the overall project design, the main system implementation, and most of the paper writing. He was primarily responsible for the prompt-layer reordering pipeline and its integration with the vLLM serving path, including the core prompt construction and request handling logic. He also ran most of the main latency experiments, including the TTFT comparison and the end-to-end evaluation, and took the lead in revising the abstract, introduction, methodology, and experiments sections.

Rong Gu. Contributed to the cache-side implementation and system validation. He worked on the main cache data structures and management logic, including the knowledge-tree organization and cache bookkeeping modules. He also helped with profiling, cache-efficiency analysis, and revision of the implementation and discussion sections.

Mingyuan Li. Contributed to workload construction, experimental evaluation, and result analysis. He helped build the QA-based and workload-diversity experiments, including the overlap-sensitivity and multi-domain settings, and assisted with result checking and revision of the analysis, findings, and conclusion sections.

A Experimental Configuration Details

This appendix lists the hardware and software settings used in the main experiments.

Table 13. Hardware and software configuration.

	Config A	Config B
GPU	4060 Ti 8 GB	4090 24 GB
Model	Qwen2.5-1.5B	Qwen2.5-7B
vLLM	0.8.5.post1	0.17.1
`max_model_len`	2048	4096
`gpu_mem_util`	0.85	0.90
Corpus	100 docs	500 docs
Queries	100	200
Top-k	5	5
Use	TTFT, overlap	All (§5–6.6)

B Overlap Sweep Raw Data

This appendix reports the raw numbers behind the overlap-sensitivity figure in the main text.

Table 14. Raw data for overlap sensitivity (RTX 4060 Ti, Qwen2.5-1.5B-Instruct).

Overlap Frac.	Jaccard	Retrieval p50 (ms)	Optimized p50 (ms)	Improv.
0.0	0.386	126.8	111.7	11.9%
0.2	0.386	128.9	114.2	11.5%
0.4	0.417	111.1	77.8	30.0%
0.6	0.526	75.5	60.5	19.9%
0.8	0.661	50.4	50.8	−0.8%

C Systems Benchmark Details

The systems analysis in Section 6.3 uses Config B (RTX 4090) with `enforce_eager=True`, so CUDA graphs are disabled for more stable measurement.

References

Agarwal, S., Sundaresan, S., Mitra, S., Mahapatra, D., Gupta, A., Sharma, R., Kapu, N. J., Yu, T., and Saini, S. Cache-Craft: Managing chunk-caches for efficient retrieval-augmented generation. *Proceedings of the ACM on Management of Data*, 3(3):136:1–136:28, 2025. doi: 10.1145/3725273. URL https://doi.org/10.1145/3725273.

Gao, Y., Xiong, Y., Gao, X., Jia, K., Pan, J., Bi, Y., Dai, Y., Sun, J., Wang, M., and Wang, H. Retrieval-augmented generation for large language models: A survey. *CoRR*, abs/2312.10997, 2023. doi: 10.48550/arXiv.2312.10997. URL https://arxiv.org/abs/2312.10997.

Jiang, Y., Huang, Y., Cheng, L., Deng, C., Sun, X., and Mai, L. ContextPilot: Fast long-context inference via context reuse, 2025. URL https://arxiv.org/abs/2511.03475.

Jin, C., Zhang, Z., Jiang, X., Liu, F., Liu, S., Liu, X., and Jin, X. RAGCache: Efficient knowledge caching for retrieval-augmented generation. *ACM Transactions on Computer Systems*, 44(1):2:1–2:27, 2026. doi: 10.1145/3768628. URL https://doi.org/10.1145/3768628.

Kwiatkowski, T., Palomaki, J., Redfield, O., Collins, M., Parikh, A., Alberti, C., Epstein, D., Polosukhin, I., Devlin, J., Lee, K., Toutanova, K., Jones, L., Kelcey, M., Chang, M.-W., Dai, A. M., Uszkoreit, J., Le, Q., and Petrov, S. Natural Questions: A benchmark for question answering research. *Transactions of the Association for Computational Linguistics*, 7:452–466, 2019. doi: 10.1162/tacl_a_00276. URL https://aclanthology.org/Q19-1026/.

Kwon, W., Li, Z., Zhuang, S., Sheng, Y., Zheng, L., Yu, C. H., Gonzalez, J. E., Zhang, H., and Stoica, I. Efficient memory management for large language model

serving with PagedAttention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, SOSP '23, pp. 611–626. Association for Computing Machinery, 2023. doi: 10.1145/3600006.3613165. URL https://doi.org/10.1145/3600006.3613165.

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., K"uttler, H., Lewis, M., Yih, W.-t., Rockt"aschel, T., Riedel, S., and Kiela, D. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, pp. 9459–9474, 2020. URL https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html.

Liu, Y., Li, H., Cheng, Y., Ray, S., Huang, Y., Zhang, Q., Du, K., Yao, J., Lu, S., Ananthanarayanan, G., Maire, M., Hoffmann, H., Holtzman, A., and Jiang, J. CacheGen: KV cache compression and streaming for fast large language model serving. In *Proceedings of the ACM SIGCOMM 2024 Conference*, ACM SIGCOMM '24, pp. 38–56. Association for Computing Machinery, 2024. doi: 10.1145/3651890.3672274. URL https://doi.org/10.1145/3651890.3672274.

Lu, S., Wang, H., Rong, Y., Chen, Z., and Tang, Y. TurboRAG: Accelerating retrieval-augmented generation with precomputed KV caches for chunked text. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pp. 6588–6601. Association for Computational Linguistics, 2025. doi: 10.18653/v1/2025.emnlp-main.334. URL https://aclanthology.org/2025.emnlp-main.334/.

Tang, Y., Wang, Z., Qu, A., Yan, Y., Wu, Z., Zhuang, D., Kai, J., Hou, K., Guo, X., Zhao, J., Zhao, Z., and Ma, W. ItiNera: Integrating spatial optimization with large language models for open-domain urban itinerary planning. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*, pp. 1413–1432, Miami, Florida, US, 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.emnlp-industry.104. URL https://aclanthology.org/2024.emnlp-industry.104/.

vLLM Project. Automatic Prefix Caching, 2025. URL https://docs.vllm.ai/en/latest/features/automatic_prefix_caching/. Accessed: 2026-04-16.

Yang, A., Yang, B., Zhang, B., Hui, B., Zheng, B., Yu, B., Li, C., Liu, D., Huang, F., Wei, H., Lin, H., Yang, J., Tu, J., Zhang, J., Yang, J., Yang, J., Zhou, J., Lin, J., Dang, K., Lu, K., Bao, K., Yang, K., Yu, L., Li, M., Xue, M., Zhang, P., Zhu, Q., Men, R., Lin, R., Li, T., Xia, T., Ren, X., Ren, X., Fan, Y., Su, Y., Zhang, Y., Wan, Y., Liu, Y., Cui, Z., Zhang, Z., and Qiu, Z. Qwen2.5 Technical Report. *CoRR*, abs/2412.15115, 2024. doi: 10.48550/arXiv.2412.15115. URL https://arxiv.org/abs/2412.15115.

Yao, J., Li, H., Liu, Y., Ray, S., Cheng, Y., Zhang, Q., Du, K., Lu, S., and Jiang, J. CacheBlend: Fast large language model serving for RAG with cached knowledge fusion. In *Proceedings of the Twentieth European Conference on Computer Systems*, EuroSys '25, pp. 94–109, Rotterdam, Netherlands, 2025. Association for Computing Machinery. doi: 10.1145/3689031.3696098. URL https://doi.org/10.1145/3689031.3696098.

Yu, D., Bao, R., Ning, R., Peng, J., Mai, G., and Zhao, L. Spatial-RAG: Spatial retrieval augmented generation for real-world geospatial reasoning questions, 2025. URL https://arxiv.org/abs/2502.18470.

Zheng, L., Yin, L., Xie, Z., Sun, C., Huang, J., Yu, C. H., Cao, S., Kozyrakis, C., Stoica, I., Gonzalez, J. E., Barrett, C. W., and Sheng, Y. SGLang: Efficient execution of structured language model programs. In *Advances in Neural Information Processing Systems*, 2024. URL https://openreview.net/forum?id=VqkAKQibpq.